JS / TS Syntax Cheatsheet

Braces, commas, colons and dots, for a C# developer reading React + TypeScript. The examples come from Driveway (and the course author's code). Start with section 1: most lines that look scary are just objects nested inside objects.

1 Reading a scary line: the routine

1. **Find the outermost bracket pair.** `( )` = arguments or parameters. `[ ]` = an array (or a by-position pattern). `{ }` = it depends on *where* it is; see section 2.
2. **Commas only split items at the current level.** A comma inside an inner `{ }` belongs to that inner object, not to the outer one.
3. **Read each item on its own.** It's one of: `key: value`, `...spread`, or a shorthand `name`.

Bracket colours by depth: `level 0` `level 1` `level 2` `level 3`

Example A: building an object as the only argument (author's `AuctionForm`)

```
const newAuction = await createListing(        // ( = argument list. It has ONE argument:
    {                                           // { in a value position = build an object
        ...data,                                // item 1: copy every property of data
        reservePrice: data.reservePrice || 0,   // item 2: overwrite reservePrice ("" becomes 0)
        // item 3: overwrite auctionEnd (local time from the input → UTC string)
        auctionEnd: new Date(data.auctionEnd).toISOString()
    }
);
```

Result: the same object as `data`, with two properties replaced. In C# this is a record `with` expression: `data with { ReservePrice = ..., AuctionEnd = ... }`.

Example B: an object whose values are objects (`features/session/actions.ts`)

```
// left of = : a PATTERN (take accessToken out).   right of = : a call with ONE object argument
const {accessToken} = await auth.api.getAccessToken({
    body: {useAccountCookie: true},       // item 1: key "body", value = ANOTHER object
    headers: await headers()               // item 2: key "headers", value = the awaited result
});
```

Example C: a nested pattern (`AuctionForm` with React Hook Form)

```
const {control, handleSubmit, setFocus, formState: {isSubmitting}} = useForm();
//      item 1   item 2        item 3    item 4: go INSIDE formState and take isSubmitting
// Variables created: control, handleSubmit, setFocus, isSubmitting. NOT formState.
```

Example D: JSX props holding an object holding objects (`AuctionForm`)

```
// 1st { = "JS goes here".   2nd { = an object literal.
rules={{
    required: "Auction end date/time is required",   // item 1
    validate: value =>                               // item 2: the value is a function (lambda)
        new Date(value) > new Date() || "Auction end date must be in the future"
}}
```

2 What does this `{` mean? Look at where it is

Position	Example	Meaning
Left of `=`, or in parameters	`const {id} = await props.params`	**Pattern**: take properties *out* of an object
Right of `=`, or as an argument	`{...data, reservePrice: 0}`	**Object literal**: *build* an object
After `type X =`, or after `:` in a type	`type Props = {label?: string}`	**Type shape**: describes an object
JSX attribute or child	`disabled={isPending}`	**Expression slot**: "JavaScript goes here"
JSX, doubled	`rules={{required: "..."}}`	Expression slot + object literal inside it
After `=>`	`() => { doThing(); }`	**Function body** (a block, as in C#)
After `=>`, wrapped in `( )`	`() => ({a: 1})`	**Returns an object**. The parentheses stop it being read as a body
After `import` / `export`	`import {useEffect} from "react"`	**Named imports**: which exports to bring in
After `if`, `for`, `function`	`if (x) { ... }`	**Block**, same as C#

3 Commas

A comma **separates items in a list**. Lists appear in: object literals, arrays, arguments, parameters, patterns, imports, generic parameters. In a type shape you may use `,` or `;`.

List	Example
Object literal	`{ method: "POST", body: json, headers: h }`
Array	`["openid", "profile", "auctionApp"]`
Arguments	`new URL("/", request.url)`
Parameters	`(key: "seller" | "winner", value: string) => ...`
Pattern (object)	`const {pageNumber, pageSize, searchTerm} = params`
Pattern (array, by position)	`const [isPending, startTransition] = useTransition()`
Imports	`import {useEffect, useTransition} from "react"`

> **A comma never ends a statement.** That's `;` (or a new line). If you see a comma, you're inside a list: find which bracket opened it.

Trailing commas are allowed: `{a, b,}`, `[1, 2,]`, `import {useEffect,}`. They're harmless, and they make diffs cleaner when you add an item later.

4 The colon `:`

Context	Example	Meaning
Object literal	`{ seller: "bob" }`	key **:** value
Pattern, then a **name**	`const {accessToken: token} = r`	**Rename**. Only `token` exists afterwards
Pattern, then `{`	`const {formState: {errors}} = form`	**Go inside**. Only `errors` exists afterwards
Type shape	`label?: string`	name **:** type (`?` = optional)
Type shape, **literal** after the colon	`{ ok: false; error: string }`	name **:** *literal type*. `ok` must be exactly `false`. Still a type, even though it looks like a value (section 9.5)
Parameter	`(data: FieldValues) => ...`	parameter type
Return type	`async function f(): Promise<Auction>`	what the function returns
Ternary	`isPending ? "Submitting..." : "Submit"`	"else"

In a pattern, the colon is never followed by a type. It's followed by a new name (rename) or another pattern (go inside). To type a pattern, put the type *after the whole pattern*:
`const {accessToken: token}: {accessToken: string} = result;`

Which world am I in? In the **type world**, everything right of a colon is a type: after `type X =`, after `:` on a variable, parameter or return, and inside `< >`. In the **value world** (right of `=` in `const` / `let`, arguments, JSX), a colon pairs a key with a value.
`{ ok: false; status: number }` (type: `ok` must be `false`, `status` must be a number) vs `{ ok: false, status: 404 }` (value: one real object).

5 Three dots `...`

Where	Name	What it does	Example
Pattern	rest	**Collects** everything not named	`const {label, ...controllerProps} = props`
Object literal	spread	**Copies** every property in	`{...controllerProps, defaultValue: ""}`
Array literal	spread	Copies every item in	`[...items, newItem]`
JSX props	spread	Passes each property as a prop	`<Input {...field} />`
Parameters	rest parameter	Collects the arguments into an array	`(...args) => ...`

With spread, later wins. `{...data, reservePrice: 0}` → `0` wins. `{reservePrice: 0, ...data}` → `data`'s value wins. So put the *overrides after* the spread.

6 Shorthands and defaults

You see	It means
`{ make, model }` (value)	`{ make: make, model: model }`: key and variable share a name
`const { make } = obj` (pattern)	`const make = obj.make`
`({callbackUrl = "/"}: Props)`	Default in a pattern: used when `callbackUrl` is `undefined` (`LoginButton`)
`const {control = formControl} = props`	Same idea: use the prop, or else `formControl` (React Hook Form internals)
`function f(x = 1)`	Default parameter, as in C#

7 Operators, with C# next to them

JS / TS	Does	C#
`a?.b`	`undefined` if `a` is null/undefined, instead of throwing	`a?.b`
`a ?? b`	`b` only if `a` is null/undefined	`a ?? b`
`a || b`	`a` if truthy, else `b`. **Returns an operand, not a bool**	no equivalent (C# `||` gives a bool)
`a && b`	`a` if falsy, else `b`	no equivalent
`{cond && <X/>}`	Render `X` or nothing	`@if (cond) { <X/> }`
`!!a`	Turn anything into `true` / `false`	roughly `a is not null`
`===` / `!==`	Equal, no type conversion (use these, not `==`)	`==` / `!=`
`` `Bearer ${token}` ``	Template string	`$"Bearer {token}"`
`c ? a : b`	Ternary	same

Falsy values: `false` `0` `""` `null` `undefined` `NaN`. Everything else is truthy, including `"undefined"`, `"0"`, `[]` and `{}`.

> **Traps you've already hit:**
> `String(undefined) || "/"` → `"undefined"` (a non-empty string is truthy).
> `{count && <Badge/>}` with `count = 0` → shows a `0` on the page. Use `count > 0 &&`.
> An object or function inside a string → `"[object Object]"` or `"function search() {...}"`.

8 Arrow functions

JS / TS	C#
`x => x * 2`	`x => x * 2`
`(a, b) => a + b`	`(a, b) => a + b`
`() => { doThing(); }`	`() => { DoThing(); }`
`async () => { await f(); }`	`async () => await FAsync()`
`() => ({ a: 1 })`	`() => new { A = 1 }`

> **An `await` pauses only the function it's written in.** An async arrow handed to something else (`startTransition`, `setTimeout`, `.forEach`) is *not* awaited by the outer function. It's like C#'s `_ = Task.Run(async () => ...)`.

9 Types

Everything here lives in the **type world**. Types are checked by the compiler and your editor, then **erased**: at run time only plain JavaScript is left. So a type can't *do* anything; it only describes what's allowed.

9.1 A type is a set of values

```
type Make  = string;                  // every possible string
type Side  = "seller" | "winner";     // only these two strings
type Flag  = false;                   // only the value false
type Props = { label?: string };      // every object whose label is a string or missing
```

Thinking in **sets of values** explains `&` and `|`. `A & B` is the **overlap**: fewer values fit, but each one guarantees *more* properties. `A | B` is **either**: more values fit, but you can only rely on what they *share*.

9.2 Intersection `&`: combine two shapes (`components/ui/app-text-input.tsx`)

```
type Props = {
    label?: string;
    placeholder?: string;
    multiline?: boolean;
} & UseControllerProps;     // from react-hook-form: name, rules, control, defaultValue, ...

// A Props value must satisfy BOTH sides:
const p: Props = {
    label: "Make of car",       // ← from your side
    name: "make",               // ← from UseControllerProps (required there)
    control,                    // ← from UseControllerProps (shorthand for control: control)
};
// Leave out name → error: Property 'name' is missing in type ...
```

C#: `interface IProps : IUseControllerProps { string? Label { get; } ... }`. One difference: TS can merge *any* two shapes, even ones nobody designed to fit together, because TS only checks the shape (it's **structural**). C# needs the class to *declare* the interface (**nominal**).

9.3 Union `|` : one of several (`features/listings/actions.ts`)

```
type ListingSearchParams = {
    searchTerm?: string | string[];   // ?searchTerm=bmw → string; repeated → array
};

let searchTerm: string | string[] = ...;

searchTerm.toUpperCase();   // ❌ arrays don't have toUpperCase
searchTerm.toString();      // ✅ both have toString (why your actions.ts compiles)

if (Array.isArray(searchTerm)) {
    searchTerm.join(",");         // TS narrowed: here it's string[]
} else {
    searchTerm.toUpperCase();     // TS narrowed: here it's string
}
```

Narrowing means a check that TypeScript understands, after which it knows which member you have: `typeof x === "string"`, `Array.isArray(x)`, `if (x)`, `x === null`, or checking a tag (9.5). Until you narrow, you may only use what *every* member has.

9.4 `| null` : "maybe nothing yet" (`features/session/AuthTestButton.tsx`)

```
const [result, setResult] = useState<{status: number, body: string} | null>(null);

result.status;                                  // ❌ 'result' is possibly 'null'
result?.status;                                 // ✅ type: number | undefined
{result && <div>HTTP {result.status}</div>}     // ✅ inside the &&, result isn't null
```

C#: `Result? result = null;` with nullable reference types on. Same warning, same fix (`?.` or a null check).

9.5 Literal types and tagged unions (`lib/fetch-wrapper.ts`)

```
export type FetchResult<T> =
      { ok: true;  data: T }                          // success shape
    | { ok: false; status: number; error: string };     // failure shape
//      ↑ ok's TYPE is the literal true or false. That literal is the "tag"

const result: FetchResult<Auction> = await createListing(values);

result.data;                                    // ❌ TS doesn't know which shape yet

if (!result.ok) {
    toast(result.status, result.error);         // ✅ narrowed to the failure shape
    return;
}
router.push(`/listings/${result.data.id}`);       // ✅ only the success shape is left here
```

`true` and `false` are also **types**, each allowing exactly one value (like `"seller"` in `"seller" | "winner"`). Because the two shapes have *different* literals in `ok`, checking `ok`

tells TS which shape you're holding. If the tag were `ok: boolean`, it couldn't tell them apart. The author's `AuctionForm` uses exactly this `if (!result.ok)` check.

```
// The same idea in C#: a small record hierarchy + pattern matching
abstract record FetchResult<T>;
record Ok<T>(T Data) : FetchResult<T>;
record Fail<T>(int Status, string Error) : FetchResult<T>;

var message = result switch {
    Ok<Auction> ok    => $"Created {ok.Data.Id}",
    Fail<Auction> f   => $"{f.Status}: {f.Error}",
};
```

9.6 Generics `<T>`: a type placeholder

```
async function fetchWrapper<T>(url: string, init: RequestInit = {}): Promise<FetchResult<T>>
//                          ↑ T is filled in by whoever calls it

const res = await fetchWrapper<Auction>(`/auctions/${id}`);
// T = Auction → res is FetchResult<Auction> → after narrowing, res.data is an Auction

const [result, setResult] = useState<Result | null>(null);   // T = Result | null
const [count, setCount]   = useState(0);                     // T inferred from 0 → number
```

Same as C#'s `Task<FetchResult<T>> FetchWrapperAsync<T>(string url)`. TS often **infers** `T` from the arguments, so you only write `<...>` when it can't guess (like `null`, which says nothing about what will come later).

9.7 Optional `?:` and types taken from values

```
type Props = { label?: string };     // reading label gives string | undefined

// lib/auth.ts: build a type FROM a value
export type SessionUser = typeof auth.$Infer.Session.user;

// Same keyword, two different things:
type T = typeof someValue;             // TYPE position: the type of this value
if (typeof x === "string") { ... }     // VALUE position: a run-time check
```

Because `SessionUser` is computed from your `auth` config, adding a user field in `auth.ts` adds it to the type automatically. Nothing else needs editing.

9.8 Summary

TS	Meaning	Closest C#
`A & B`	**Intersection**: values that are both, so they have *all* properties of A and B	interface inheritance `: IA, IB`
`A | B`	**Union**: either one. Narrow (`if`) before using what isn't shared	base type + pattern matching
`"seller" | "winner"`	Only these exact strings	an `enum`, roughly
`{ok: true; data: T} | {ok: false; ...}`	**Tagged union**: check the literal tag to narrow	record hierarchy + `switch`
`label?: string`	Optional property (may be `undefined`)	`string? Label`
`useState<Result | null>(null)`	Generic type argument	`new List<T>()`
`typeof auth.$Infer.Session.user`	A type taken *from a value*	none

10 C# ↔ JS/TS quick map

C#	JS / TS
`DateTime.Now`	`new Date()` (always date *and* time; there's no `Today`)
`DateTime.Parse("2026-10-25T18:00")`	`new Date("2026-10-25T18:00")` (no zone → read as **local** time)
`d.ToUniversalTime().ToString("o")`	`d.toISOString()` (always UTC, ends in `Z`)
`s[17..]` / `s.Substring(17)`	`s.slice(17)`
`s[..5]`	`s.slice(0, 5)`
`if (x is { } v)`	`if (x)`: TS narrows `T | undefined` to `T` inside
`record with { A = 1 }`	`{ ...obj, a: 1 }`
`var (a, b) = tuple;`	`const [a, b] = array;` (by position)
`$"Hi {name}"`	`` `Hi ${name}` ``
`_ = Task.Run(async () => ...)`	`startTransition(async () => ...)` (nobody awaits it)